

Отчёт по лабораторной работе №0

University: ITMO University
Faculty: FICT
Course: Введение в веб технологии
Year: 2025/2026
Group: U4225
Author: Арланова Алина Андреевна
Lab: Lab0 — Создание репозитория и настройка рабочего окружения
Date of create: 14.09.2026
Date of finished: 18.09.2026

Цель работы

Научиться создавать репозитории, настраивать рабочее окружение и освоить основы работы с Git и GitHub: аутентификацию по SSH, клонирование, ветвление, коммиты, Pull Request и слияние изменений.

Рабочее окружение

Компонент	Значение
Операционная система	macOS (Apple Silicon, arm64)
Оболочка	zsh
Система контроля версий	Git
Хостинг репозитория	GitHub
Аутентификация	SSH, ключ Ed25519
Локальный путь репозитория	~/Documents/GitHub/devops-lab-arlanova
Удалённый репозиторий	git@github.com:petrunina19257-hub/devops-lab-arlanova.git

Ход работы

1. Настройка Git

Учётная запись GitHub — petrunina19257-hub . Проверена и задана идентификация автора коммитов:

```
git config --global user.name "petrunina19257-hub"
git config --global user.email "petrunina19257@gmail.com"
git config --global init.defaultBranch main
```

Параметр init.defaultBranch main задаёт имя стартовой ветки в новых репозиториях.

2. Настройка SSH-ключей

Проверено отсутствие существующих ключей:

```
ls -la ~/.ssh
```

```
ls: /Users/alina/.ssh: No such file or directory
```

Сгенерирована пара ключей по алгоритму Ed25519 (парольная фраза не задавалась):

```
ssh-keygen -t ed25519 -C "petrunina19257@gmail.com"
```

Ключ добавлен в ssh-agent и в Связку ключей macOS, чтобы сохраняться между перезагрузками:

```
eval "$(ssh-agent -s)"
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

```
Identity added: /Users/alina/.ssh/id_ed25519 (petrunina19257@gmail.com)
```

Публичный ключ скопирован в буфер обмена и добавлен в настройках GitHub
(*Settings* → *SSH and GPG keys* → *New SSH key*):

```
pbcopy < ~/.ssh/id_ed25519.pub
```

Проверка соединения:

```
ssh -T git@github.com
```

```
Hi petrunina19257-hub! You've successfully authenticated, but GitHub does not provide shell access.
```

Ответ подтверждает, что GitHub опознаёт пользователя по ключу. Приватный ключ `~/.ssh/id_ed25519` остаётся только на локальной машине; на сервер передаётся исключительно публичная часть `id_ed25519.pub`.

3. Создание репозитория

На GitHub создан репозиторий `devops-lab-arlanova` — имя сформировано по шаблону `devops-lab-[фамилия]`. При создании включена опция *Add a README file*: она формирует первый коммит `a7016de Initial commit` и ветку `main`. Файлы `.gitignore` и `LICENSE` на этом шаге не добавлялись — они создаются вручную как отдельные пункты работы.

4. Клонирование репозитория

```
cd ~/Documents/GitHub
git clone git@github.com:petrunina19257-hub/devops-lab-arlanova.git
cd devops-lab-arlanova
```

```
Cloning into 'devops-lab-arlanova'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.
```

Использован SSH-адрес, поэтому пароль или токен не запрашивались. Проверка привязки к удалённому репозиторию:

```
git remote -v
```

```
origin  git@github.com:petrunina19257-hub/devops-lab-arlanova.git (fetch)
origin  git@github.com:petrunina19257-hub/devops-lab-arlanova.git (push)
```

Отдельно отмечу возникшую проблему: первая попытка клонирования завершилась ошибкой

```
fatal: unable to get current working directory: Operation not permitted
```

Причина не в Git, а в механизме защиты macOS: приложению «Терминал» не был выдан доступ к папке «Документы». Решение — *Системные настройки* → *Конфиденциальность и безопасность* → *Файлы и папки* → *Терминал* → *Папка «Документы»*, после чего Терминал требуется перезапустить полностью, так как разрешения применяются только к новому процессу.

5. Файл README.md

Заготовка, созданная GitHub, заменена полным описанием проекта. `README.md` содержит:

- название и назначение проекта;
- контактные данные автора (GitHub-профиль и ссылка на Issues проекта);
- план изучения DevOps из 10 этапов в виде чек-листа: Git и GitHub → Linux и командная строка → сети → Bash и Python → Docker и Docker Compose → CI/CD и GitHub Actions → Infrastructure as Code → Kubernetes → мониторинг и логи → итоговый проект. Для каждого этапа указана практическая задача.

6. Файл .gitignore

Создан `.gitignore` со стандартными исключениями для macOS, а также для секретов и временных файлов:

```
# macOS
.DS_Store
.AppleDouble
.LS0override
._*
.Spotlight-V100
.Trashes
.fseventsd
.TemporaryItems
.VolumeIcon.icns
.com.apple.timemachine.donotpresent

# Локальные секреты и настройки окружения
.env
.env.*
!.env.example

# Временные файлы и журналы
*.log
*.tmp
*.swp
*.swo
*~
```

Служебные файлы macOS (`.DS_Store` , ресурсные форки `._*` , индекс Spotlight) создаются системой автоматически и к проекту не относятся, поэтому в историю попадать не должны. Отдельно исключены `.env` и журналы — это защита от случайной публикации паролей и токенов. Шаблон `.env.example` выведен из исключения строкой `!.env.example` , чтобы пример конфигурации остался в репозитории.

7. Создание ветки develop

```
git checkout -b develop
```

```
Switched to a new branch 'develop'
```

Флаг `-b` создаёт ветку и сразу переключается на неё. Проверка:

```
git branch
```

```
* develop
main
```

8. Файл CONTRIBUTING.md

В ветке `develop` создан `CONTRIBUTING.md` с правилами участия в проекте:

- порядок предложения изменений — проверка существующих Issues и Pull Request, создание Issue для крупных изменений, работа через fork при отсутствии прав записи, создание отдельной ветки под задачу;
- требования к материалам — понятные объяснения, указание инструментов и шагов запуска, запрет на добавление паролей, токенов и приватных ключей;
- правила коммитов и Pull Request — небольшие коммиты с понятным описанием, описание изменений и способа проверки в PR, удаление рабочей ветки после слияния;
- канал общения — Issues проекта.

9. Коммит и отправка изменений

```
git add .
git commit -m "Initial project setup"
git push -u origin develop
```

```
[develop 924672f] Initial project setup
3 files changed
create mode 100644 .gitignore
create mode 100644 CONTRIBUTING.md
```

```
To github.com:petrunina19257-hub/devops-lab-arlanova.git
* [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
```

Флаг `-u` связывает локальную ветку с удалённой, после чего достаточно короткой команды `git push`. История коммитов:

```
git log --oneline
```

```
924672f Initial project setup
a7016de Initial commit
```

10. Создание Pull Request

В веб-интерфейсе GitHub создан Pull Request **#1 «Initial project setup»** из ветки `develop` в `main`. В описании указано:

- **что сделано** — первоначальная настройка учебного репозитория;
- **внесённые изменения** — `README.md` с планом изучения DevOps, `.gitignore` с исключениями для macOS и секретов, `CONTRIBUTING.md` с правилами участия;
- **как проверено** — рабочее дерево чистое, ветка синхронизирована с `origin/develop`, сравнение `main...develop` показывает 1 коммит и 3 изменённых файла, конфликтов нет;
- **что дальше** — удаление ветки `develop` после слияния.

GitHub подтвердил возможность автоматического слияния: «*No conflicts with base branch. Merging can be performed automatically.*».

11. Слияние Pull Request и удаление ветки

Pull Request смержен в `main` стратегией *Create a merge commit*. Создан merge-коммит `2e6085a Merge pull request #1 from petrunina19257-hub/develop`, Pull Request получил статус **Merged**. Сразу после слияния удалённая ветка `develop` удалена кнопкой *Delete branch*.

Синхронизация локального репозитория и удаление локальной ветки:

```
git checkout main
git pull
git branch -d develop
```

```
Deleted branch develop (was 924672f).
```

Строчный флаг `-d` удаляет ветку только в том случае, если её изменения уже слиты, — это защита от потери работы.

После `git pull` в списке ветвей остаётся устаревшая ссылка `remotes/origin/develop`: `git pull` не удаляет ссылки на исчезнувшие удалённые ветки. Очищается явно:

```
git fetch --prune
```

Итоговое состояние:

```
git branch -a
```

```
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
```

Ветка `develop` отсутствует и локально, и на сервере, а её изменения находятся в `main`:

```
git log --oneline --graph --decorate
```

```
*    2e6085a (HEAD -> main, origin/main) Merge pull request #1 from petrunina19257-hub/develop
|\
| * 924672f Initial project setup
|/
* a7016de Initial commit
```

12. Приведение репозитория к требованиям оформления

Согласно правилам оформления отчётов в репозиторий добавлены:

- `LICENSE` — лицензия MIT;
- каталоги `lab0`, `lab1`, `lab2`, `lab3` по количеству лабораторных работ;
- `lab0/lab0_report.md` — настоящий отчёт и его PDF-версия `lab0/lab0_report.pdf` для печати.

Пустые каталоги Git не отслеживает, поэтому в `lab1`–`lab3` добавлены файлы-заглушки `.gitkeep`.

Структура репозитория

```
devops-lab-arlanova/
├── .gitignore
├── CONTRIBUTING.md
├── LICENSE
├── README.md
├── lab0/
│   ├── lab0_report.md
│   └── lab0_report.pdf
├── lab1/
├── lab2/
└── lab3/
```

Ответы на контрольные вопросы

Зачем нужна отдельная ветка вместо работы напрямую в `main` ?

Ветка изолирует незавершённую работу: `main` всё время остаётся в рабочем состоянии, а изменения попадают в неё только после проверки. Это же даёт точку для code review — Pull Request показывает ровно то, что добавляет ветка.

Что такое Pull Request и чем он отличается от `git merge` ?

`git merge` — локальная операция слияния. Pull Request — запрос на слияние в веб-интерфейсе: он фиксирует автора, описание, диф, историю обсуждения и результаты автоматических проверок. То есть PR добавляет к слиянию процесс review и документирование причин изменения.

Почему удаляют ветку после слияния?

Её изменения уже в `main`, поэтому ветка становится мёртвым указателем. Удаление держит список ветвей коротким: в нём остаётся только активная работа.

Чем SSH-доступ удобнее HTTPS?

При SSH аутентификация идёт по ключевой паре: пароль или токен при каждом `push` вводить не нужно, приватный ключ не покидает компьютер. HTTPS требует персонального токена и его хранения в менеджере учётных данных. В этой работе разница видна напрямую: после настройки ключа ни `clone`, ни `push` не запрашивали учётные данные.

Зачем нужен `.gitignore` ?

Он не даёт попасть в историю тому, что не относится к проекту: служебным файлам ОС и редактора, артефактам сборки и, главное, секретам. Удалить секрет из истории Git после публикации трудно — файл остаётся в предыдущих коммитах, поэтому дешевле исключить его заранее.

Вывод

В ходе работы настроено рабочее окружение для дальнейшего изучения DevOps и пройден полный базовый цикл работы с Git и GitHub: настройка аутентификации по SSH-ключу, создание и клонирование репозитория, ведение файлов документации, работа в отдельной ветке, коммит, отправка изменений на сервер, создание Pull Request с описанием, слияние и удаление отработавшей ветки.

Отдельно стоит отметить три практических вывода. Во-первых, схема «ветка → Pull Request → слияние» полезна даже в одиночном учебном проекте: описание PR фиксирует, что и зачем менялось, и через месяц история изменений читается без усилий. Во-вторых, `.gitignore` и запрет на коммит секретов нужно настраивать до первого коммита, а не после — история Git хранит все прошлые версии файлов, и постфактум убрать оттуда лишнее существенно сложнее. В-третьих, ошибка `Operation not permitted` при клонировании показала, что часть проблем возникает не в Git, а в окружении: прежде чем искать причину в командах, стоит проверить права доступа операционной системы.

Полученные навыки — основа для следующих работ: контейнеризации, настройки CI/CD и автоматизации развёртывания.